Universidad de Costa Rica

Sede de Occidente, Recinto de Tacares

IF0009 Desarrollo de Software VI

Segundo semestre 2026

Proyecto de investigación: exploración de tecnologías modernas de desarrollo

Catálogo de productos con Supabase

Guía de aprendizaje y documentación del proyecto

Tecnología investigada: Supabase

Grupo 6

Integrantes

Diego Zamora Arce, C5L139

Rafael Porras Rakow, C5I449

Jered Arias Avila, C5C706

Docente: Angélica María Ulate Céspedes

Fecha de entrega: 10 de noviembre de 2026

1. Introducción

Supabase es una plataforma que le entrega a una aplicación una base de datos PostgreSQL, autenticación de usuarios, almacenamiento de archivos y una API, sin que el equipo tenga que montar ni mantener un servidor propio. La escogimos como tema porque resuelve una parte del desarrollo que en los cursos casi siempre hacemos a mano: el backend que guarda los datos y los entrega a la interfaz.

Para probarla construimos un catálogo de productos. Es una página web de un solo archivo que consulta una tabla de Supabase, muestra los productos y permite agregar nuevos desde un formulario. No hace falta instalar Node.js ni otras dependencias, porque la librería de Supabase se carga desde un CDN.

Quien siga esta guía va a crear un proyecto en Supabase, definir una tabla con SQL, darle permisos y políticas de seguridad, y conectarla a una página que lee e inserta datos por medio de la API de Supabase. Las secciones 3 a 5 son el tutorial; la sección 2 explica la tecnología, y el código completo está en el repositorio indicado en la sección 9.

2. Investigación

2.1 Qué es Supabase

Supabase es una plataforma de backend como servicio (BaaS, *backend as a service*). No es un lenguaje ni un framework ni una biblioteca: es un servicio en la nube que se administra desde un panel web y que la aplicación usa por medio de librerías cliente. En su repositorio oficial el equipo la presenta como una plataforma de desarrollo construida sobre Postgres, que ofrece funciones parecidas a las de Firebase con herramientas de código abierto (Supabase, s. f.-g).

El backend es la parte de una aplicación que procesa los datos, los almacena y se encarga de su seguridad (Kakashvili, 2023). Con Supabase esa parte ya viene armada. Cada proyecto trae una base de datos PostgreSQL dedicada y una API que se genera sola a partir de las tablas que uno crea, de modo que la aplicación puede leer y escribir datos directamente desde el navegador (Supabase, s. f.-c).

Sirve para aplicaciones web, móviles y de escritorio. Tiene librerías cliente oficiales para JavaScript, Flutter y Swift, y una para Python que todavía está en beta (Supabase, s. f.-h).

2.2 Origen

Supabase fue fundada en 2020 por Paul Copplestone, su director ejecutivo, y Ant Wilson, su director de tecnología (Bort, 2025). La necesidad que buscaba cubrir era la de tener un backend listo para usar, como Firebase, pero sobre una base de datos relacional y con herramientas abiertas. Su documentación aclara que no pretende ser una copia uno a uno de Firebase y que eligió Postgres en lugar de una base de datos NoSQL (Supabase, s. f.-b).

La plataforma pasó sus primeros años con la etiqueta beta. El 15 de abril de 2024 anunció su disponibilidad general; en ese anuncio el equipo explica que el producto ya estaba listo para producción desde antes y que conservó la etiqueta hasta asegurarse de poder atender a cualquier tipo de cliente (Supabase, 2024). En octubre de 2025 la empresa recibió 100 millones de dólares con una valoración de 5000 millones y reportaba 4 millones de desarrolladores como usuarios (Bort, 2025).

Con los años pasó de ser una base de datos con API a un conjunto de servicios: autenticación con correo, teléfono, redes sociales y enlaces mágicos, almacenamiento de archivos, funciones que corren en el servidor (Edge Functions), mensajes en tiempo real y búsqueda vectorial para aplicaciones de inteligencia artificial (Supabase, s. f.-h). El motor de fondo es mucho más viejo: PostgreSQL viene del proyecto POSTGRES de la Universidad de California en Berkeley, iniciado en 1986 (The PostgreSQL Global Development Group, s. f.-a).

El cambio más reciente afecta directamente esta guía. Supabase anunció que desde el 30 de octubre de 2026 las tablas nuevas dejan de quedar disponibles en la API de forma automática (Supabase, 2026). Al crear un proyecto, el formulario ahora muestra la opción *Automatically expose new tables* y recomienda desmarcarla. Con esa opción desmarcada, cada tabla nueva necesita un permiso explícito con la instrucción grant, un paso que los tutoriales escritos antes de esa fecha no incluyen.

2.3 Cómo funciona

Un proyecto de Supabase es un grupo de servicios que trabajan alrededor de una base de datos PostgreSQL. La documentación de arquitectura los enumera así (Supabase, s. f.-b):

Servicio	Qué hace
Postgres	La base de datos. Supabase no la esconde: se puede usar con todos los privilegios.
Studio	El panel web desde el que se crean tablas, se ejecuta SQL y se configura el proyecto.
PostgREST	Convierte las tablas de la base de datos en una API REST.
GoTrue (Auth)	Maneja usuarios y emite tokens JWT para el inicio de sesión.
Realtime	Envía por WebSocket los cambios de la base de datos y mensajes entre usuarios.
Storage	Guarda archivos de forma compatible con S3 y registra sus datos en Postgres.
Edge Functions	Ejecuta código JavaScript o TypeScript en el servidor con Deno.
Supavisor	Administra las conexiones a la base de datos.
Envoy	La puerta de entrada (API gateway) que recibe todas las peticiones y revisa la llave.

Para el catálogo solo intervienen tres piezas: Postgres, PostgREST y el gateway. El recorrido de una consulta es este:

```
Página (index.html + supabase-js)
  │ GET https://<proyecto>.supabase.co/rest/v1/productos
  ▼
Gateway (revisa la llave de API)
  ▼
PostgREST (convierte la petición en una instrucción SQL)
  ▼
PostgreSQL (aplica permisos y políticas RLS, ejecuta la consulta)
  ▼
```

Respuesta en JSON con las filas de la tabla productos

La librería supabase-js arma la petición. Cuando el código escribe cliente.from('productos').select('*'), la librería llama a la dirección /rest/v1/productos del proyecto, y PostgREST traduce cada petición a una sola instrucción SQL (Supabase, s. f.-c).

La seguridad tiene tres capas, y entenderlas evita casi todos los errores de la práctica:

- La llave de API. La *publishable key* (antes llamada *anon key*) se puede poner en una página web porque tiene privilegios bajos. La *secret key* (antes *service_role*) se salta todas las políticas de seguridad y nunca debe ir en el navegador (Supabase, s. f.-a). Las peticiones hechas con la publishable key entran a la base de datos con el rol anon.
- Los permisos del rol. En un proyecto que no expone las tablas nuevas de forma automática, una tabla no es accesible por la API hasta que se ejecuta un grant para el rol que la va a usar. Sin él, la API responde con el error 42501, *permission denied for table* (Supabase, 2026).
- Las políticas RLS (*Row Level Security*). Son reglas de PostgreSQL que deciden qué filas puede ver o modificar cada rol. Si RLS está activo y no existe ninguna política, se aplica una negación por defecto y no se ve ni se modifica ninguna fila (The PostgreSQL Global Development Group, s. f.-b). Las tablas creadas desde el panel traen RLS activo; las creadas con SQL hay que activarlas a mano (Supabase, s. f.-f).

2.4 Características principales

Base de datos relacional completa. Se trabaja con tablas, llaves primarias, restricciones y SQL estándar de PostgreSQL, con acceso total al motor (Supabase, s. f.-b).

API generada automáticamente. Cada tabla expuesta tiene su API REST sin escribir código de servidor. También existe una API GraphQL por medio de la extensión pg_graphql (Supabase, s. f.-c).

Seguridad dentro de la base de datos. Los permisos no viven en un servidor intermedio sino en PostgreSQL, con grant y políticas RLS. Por eso la página puede hablar directo con la base de datos sin exponer más de lo permitido.

Servicios independientes pero integrados. Entre sus principios de producto la documentación indica que cada herramienta funciona por separado, que todas se integran entre sí y que todo es

portable (Supabase, s. f.-b). En el catálogo usamos solo la base de datos y la API, sin tocar Auth ni Storage.

Código abierto. El repositorio principal se publica con licencia Apache 2.0 (Supabase, s. f.-g).

Panel web (Studio). Incluye un SQL Editor, un Table Editor para ver y editar filas, plantillas de políticas y un asesor de seguridad y rendimiento (Supabase, s. f.-h).

2.5 Qué se necesita para utilizarla

Elemento	Para este proyecto
Software necesario	Un navegador actualizado: Chrome, Edge o Firefox.
Herramientas de desarrollo	El panel web de Supabase (Studio) y las herramientas de desarrollador del navegador (tecla F12).
Lenguajes	SQL para crear la tabla y JavaScript para la página.
Editor recomendado	Visual Studio Code. La extensión Live Server es opcional.
Dependencias	La librería @supabase/supabase-js versión 2, cargada desde el CDN jsDelivr como indica la documentación (Supabase, s. f.-d). La guía se probó con la versión 2.118.0.
Requisitos de instalación	Ninguno adicional. Hace falta conexión a internet porque la librería y la base de datos están en la nube.
Cuentas	Una cuenta gratuita en supabase.com. El plan gratuito permite 2 proyectos activos y 500 MB de base de datos, y pausa los proyectos después de una semana sin uso (Supabase, s. f.-i).
Opcional para proyectos más grandes	Node.js y npm para instalar la librería con npm install @supabase/supabase-js, y la CLI de Supabase para desarrollo local (Supabase, s. f.-h). No se usan en esta guía.

2.6 Ventajas y desventajas

Ventajas

1. Es PostgreSQL de verdad. Lo que se aprende de SQL sirve fuera de Supabase, y se pueden usar relaciones entre tablas, restricciones y consultas complejas (Supabase, s. f.-b).
2. No hay que programar el servidor. Para el catálogo no escribimos ni una línea de backend: la API sale de la tabla (Supabase, s. f.-c).
3. El plan gratuito alcanza para aprender y para prototipos pequeños (Supabase, s. f.-i).
4. Es de código abierto, así que se puede revisar cómo funciona y no se depende por completo de una sola empresa (Supabase, s. f.-g).

Desventajas o limitaciones

1. La seguridad depende de configurarla bien. Una tabla expuesta sin RLS puede ser leída y modificada por cualquier rol con permisos (Supabase, s. f.-e). Al revés, una tabla con RLS y sin políticas no devuelve nada, y al principio eso parece un error del código.
2. La plataforma cambia rápido y esos cambios rompen tutoriales. El cambio de permisos del 30 de octubre de 2026 obliga a agregar un grant que antes no hacía falta (Supabase, 2026), y las llaves *anon* y *service_role* se dejarán de usar a finales de 2026 (Supabase, s. f.-a).
3. El plan gratuito tiene límites: dos proyectos activos y la pausa automática tras una semana de inactividad (Supabase, s. f.-i).
4. Exige saber SQL y bases de datos relacionales. Quien venga solo de frontend tiene que aprender eso primero.

2.7 Casos de uso

Supabase encaja en aplicaciones que necesitan guardar datos y usuarios sin un equipo dedicado al backend. Algunos ejemplos concretos:

- El inventario de un negocio pequeño, como una soda o una pulpería, que registra productos y precios y los consulta desde el celular. Es una versión ampliada de nuestro catálogo.

- Una aplicación universitaria con inicio de sesión donde cada estudiante ve solo sus propios registros. Supabase Auth maneja las cuentas y una política RLS limita las filas con auth.uid(), como en el ejemplo de su propia documentación (Supabase, 2026).
- Un chat o un tablero colaborativo que muestra cambios al instante con Realtime, que transmite mensajes, presencia de usuarios y cambios de la base de datos (Supabase, s. f.-h).
- Prototipos y productos mínimos para una feria o un hackatón, donde importa tener datos reales funcionando en horas y no en semanas.

3. Preparación del ambiente

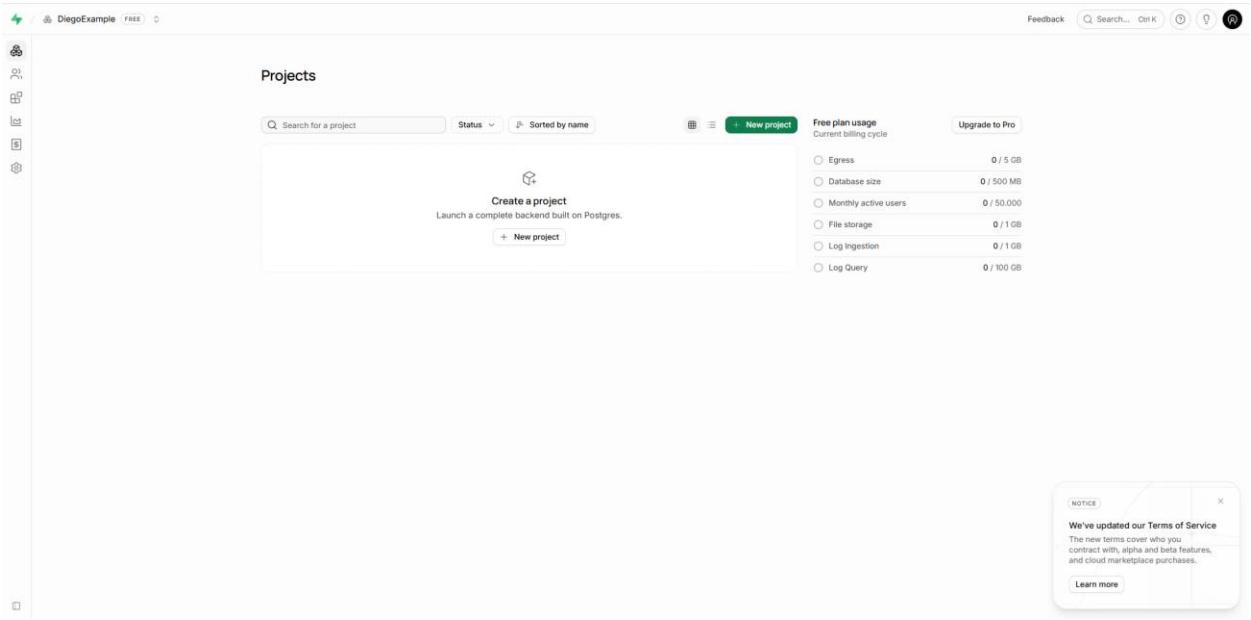
Estos pasos se hacen una sola vez y antes de la práctica. Ninguno requiere permisos de administrador en la computadora.

Paso A. Crear una cuenta en Supabase

Dónde: https://supabase.com

Cómo: clic en *Start your project*. La forma más rápida es *Continue with GitHub*. Si se registra con correo, Supabase envía un mensaje de confirmación que hay que abrir antes de continuar.

Cómo comprobarlo: al entrar a https://supabase.com/dashboard aparece la lista de proyectos, vacía si la cuenta es nueva.



Paso B. Instalar Visual Studio Code

Dónde: https://code.visualstudio.com. Cualquier versión reciente funciona; también sirve otro editor de texto.

Cómo: descargar el instalador para Windows y ejecutarlo con las opciones por defecto.

Cómo comprobarlo: abrir una terminal y ejecutar el siguiente comando, que debe mostrar el número de versión:

```
code --version
```

Paso C. Tener un navegador actualizado

Chrome, Edge o Firefox en su versión actual. La página usa JavaScript moderno (async/await) que los tres soportan.

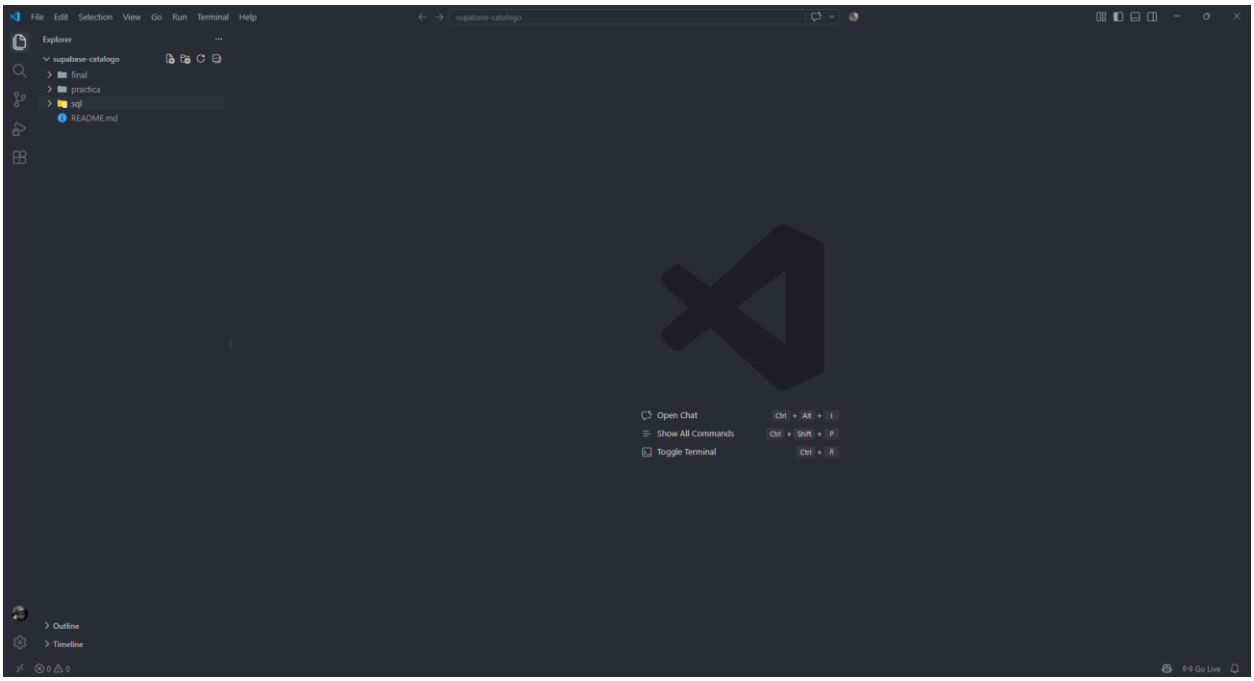
Paso D. Descargar el repositorio

Dónde: https://github.com/diego0226/proyecto2-DevIV

Cómo: botón verde *Code* → *Download ZIP*. Descomprimir el archivo y, en VS Code, usar *File* → *Open Folder* para abrir la carpeta supabase-catalogo. Quien tenga Git puede clonarlo:

```
git clone https://github.com/diego0226/proyecto2-DevIV.git
```

Resultado esperado: en el explorador de VS Code se ven las carpetas sql, practica y final, y el archivo README.md.



4. Creación del proyecto en Supabase

Paso 1. Crear el proyecto

Qué se va a hacer: crear el proyecto de Supabase, que incluye la base de datos PostgreSQL y la API.

Dónde: https://supabase.com/dashboard → botón *New project*.

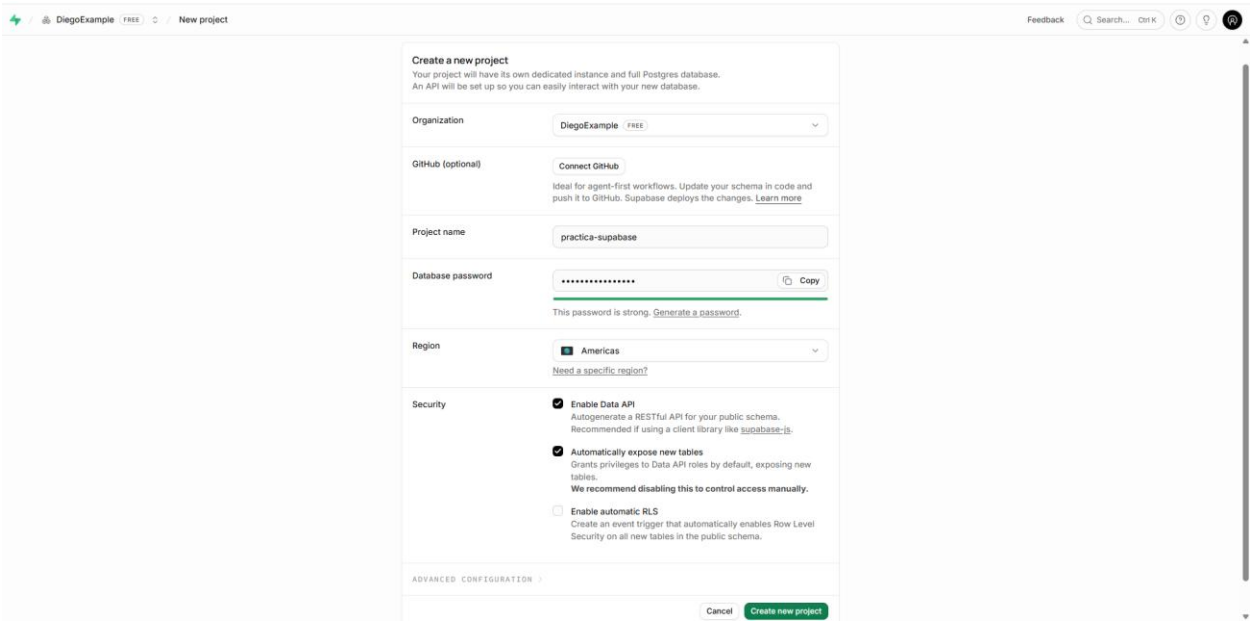
Opciones a seleccionar:

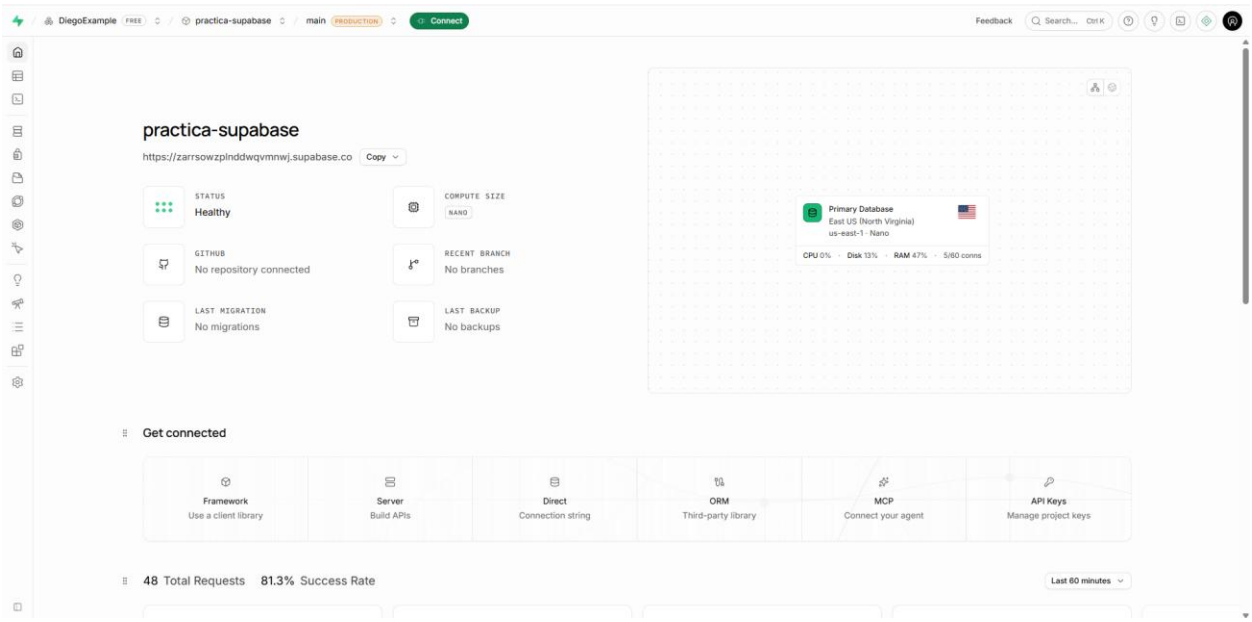
- *Organization*: la organización personal que se creó con la cuenta.
- *Project name*: practica-supabase.
- *Database Password*: clic en *Generate a password*. Esta guía no la vuelve a usar, pero conviene guardarla.
- *Region*: *Americas*, que es la opción que aparece por defecto. Una región cercana responde más rápido.
- *Security*: dejar marcada *Enable Data API*, que crea la API REST que usa la página. La opción *Automatically expose new tables* puede quedar como venga: Supabase recomienda desmarcarla, y el script del paso 2 funciona en los dos casos porque incluye el grant. *Enable automatic RLS* puede quedar desmarcada, porque el script activa RLS por su cuenta.

Por último, clic en *Create new project*.

Qué hace: Supabase reserva un servidor con PostgreSQL y levanta los servicios descritos en la sección 2.3. Tarda uno o dos minutos; mientras tanto el panel muestra que el proyecto se está configurando.

Resultado esperado: el panel del proyecto practica-supabase con el menú lateral (Table Editor, SQL Editor y los demás).





Si la cuenta ya tiene dos proyectos activos, Supabase no deja crear un tercero en el plan gratuito. En ese caso hay que pausar uno de los existentes desde su configuración.

5. Desarrollo paso a paso

El archivo practica/index.html ya trae el HTML del formulario y de la tabla, los estilos y dos funciones de apoyo: pintarTabla, que dibuja las filas, y mostrarError, que muestra los errores en un recuadro rojo. Lo que falta es justamente lo que tiene que ver con Supabase, marcado con comentarios TODO. El archivo final/index.html tiene todo resuelto y sirve para comparar.

Paso 2. Crear la tabla con SQL

Qué se va a hacer: crear la tabla productos, darle permiso a la página, activar la seguridad por filas e insertar tres productos de ejemplo.

Dónde: en el panel del proyecto, menú lateral → *SQL Editor* → *New query*.

Archivo: sql/01_crear_tabla.sql del repositorio. Copiar todo su contenido, pegarlo en el editor y hacer clic en *Run*.

Código:

```
-- ============================================================
-- Catálogo de productos · Grupo 6 · Supabase
-- Ejecutar completo en: Supabase → SQL Editor → New query → Run
-- ============================================================

-- 1. Crear la tabla (PostgreSQL)
--    id: número que PostgreSQL genera solo (1, 2, 3, ...) y es la llave primaria
--    numeric(10,2): número con hasta 2 decimales, ideal para precios
create table productos (
  id        bigint generated always as identity primary key,
  nombre    text not null,
  precio    numeric(10,2) not null check (precio >= 0),
  categoria text not null
);

-- 2. Dar permiso a la página (rol "anon") para leer e insertar en la tabla.
--    Desde el 30 de octubre de 2026 Supabase ya no expone las tablas nuevas a la API
```

```
-- de forma automática: sin este grant la página recibe el error 42501
-- "permission denied for table productos".
grant select, insert on public.productos to anon;

-- 3. Seguridad: activar RLS (Row Level Security)
-- Con RLS activo nadie puede leer ni escribir hasta que una política lo permita.
alter table productos enable row level security;

-- Permitir que la página (rol "anon", el de la publishable key) lea productos
create policy "Leer productos" on productos
  for select to anon using (true);

-- Permitir que la página agregue productos
-- (en un sistema real esto se limita a usuarios autenticados)
create policy "Agregar productos" on productos
  for insert to anon with check (true);

-- 4. Datos de ejemplo (precios en colones)
insert into productos (nombre, precio, categoria) values
  ('Café chorreado', 1200, 'Bebidas'),
  ('Gallo pinto',    2500, 'Comidas'),
  ('Tres leches',    1800, 'Postres');

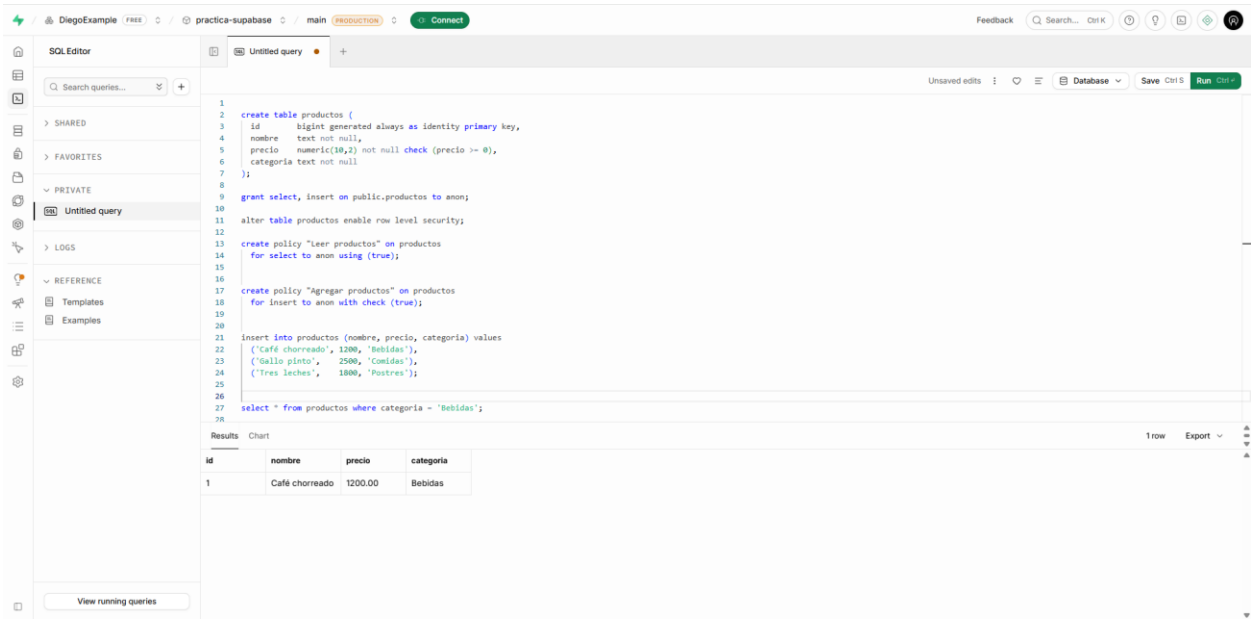
-- 5. Consulta de prueba: el resultado aparece abajo del editor
select * from productos where categoria = 'Bebidas';
```

Qué hace cada parte:

- create table define la tabla. id es un número que PostgreSQL genera solo y que funciona como llave primaria; numeric(10,2) guarda el precio con dos decimales, y check (precio >= 0) rechaza precios negativos.

- grant select, insert ... to anon le permite al rol anon, que es el de la página, consultar e insertar. Si el proyecto no expone las tablas nuevas automáticamente y falta esta línea, la API responde con el error 42501 (Supabase, 2026).
- enable row level security activa RLS. Desde ese momento nada se puede leer ni escribir por la API hasta que una política lo permita (Supabase, s. f.-e).
- Las dos create policy permiten leer (for select) y agregar (for insert) a cualquier visitante. En un sistema real se limitaría a usuarios autenticados; aquí lo dejamos abierto para aprender.
- insert into agrega tres productos con precios en colones, y el select ... where final comprueba que la tabla responde a una consulta con condición.

Resultado esperado: debajo del editor aparecen los resultados de la última consulta: una fila con Café chorreado, 1200.00, Bebidas.

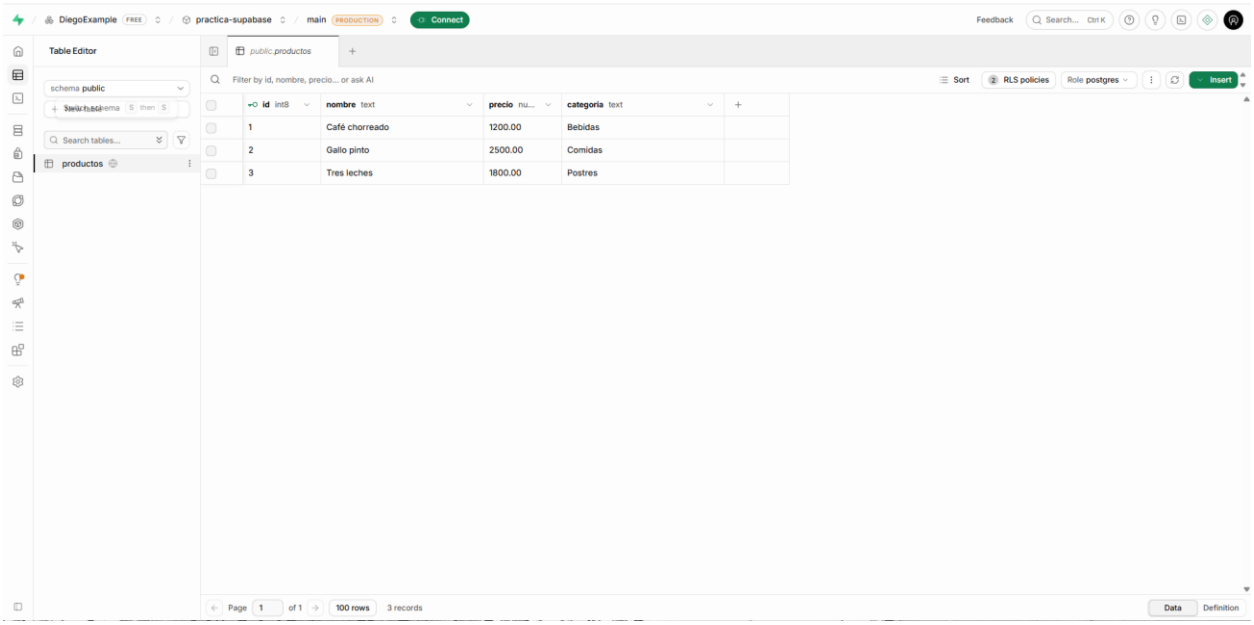


Paso 3. Revisar la tabla en el Table Editor

Qué se va a hacer: confirmar que la tabla y sus datos existen.

Dónde: menú lateral → *Table Editor* → tabla productos.

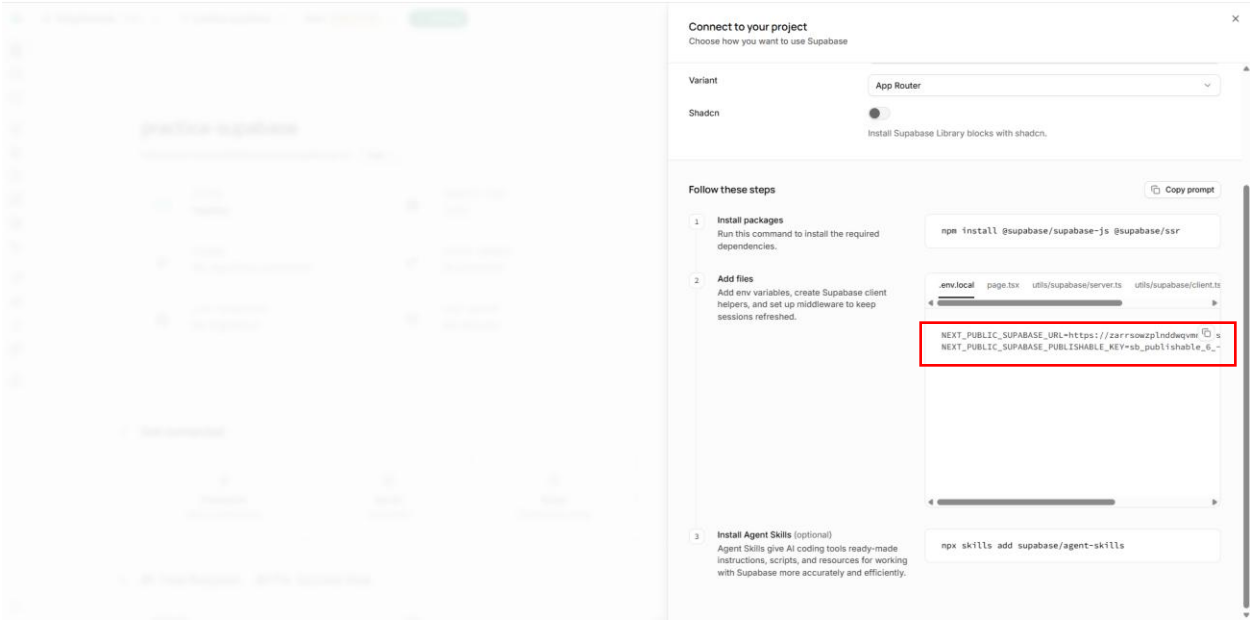
Resultado esperado: tres filas con los id 1, 2 y 3.



Paso 4. Conectar la página con el proyecto

Qué se va a hacer: copiar la dirección del proyecto y su llave pública dentro de la página.

Dónde: la *Project URL* aparece en la página principal del proyecto, debajo del nombre, con un botón *Copy*. También se puede armar con el código del proyecto que se ve en la barra de direcciones del navegador: https://<código>.supabase.co. La *publishable key* está en *Project Settings* (ícono de engranaje del menú lateral) → *API Keys* → sección *Publishable key*, con el botón de copiar. Si el proyecto muestra *anon key*, usar esa. El panel *Connect* también muestra los dos valores, pero sus instrucciones de npm y Next.js no aplican a esta guía.



Archivo: practica/index.html, abierto en VS Code. Buscar el bloque marcado como paso 4 y reemplazar los dos valores de ejemplo:

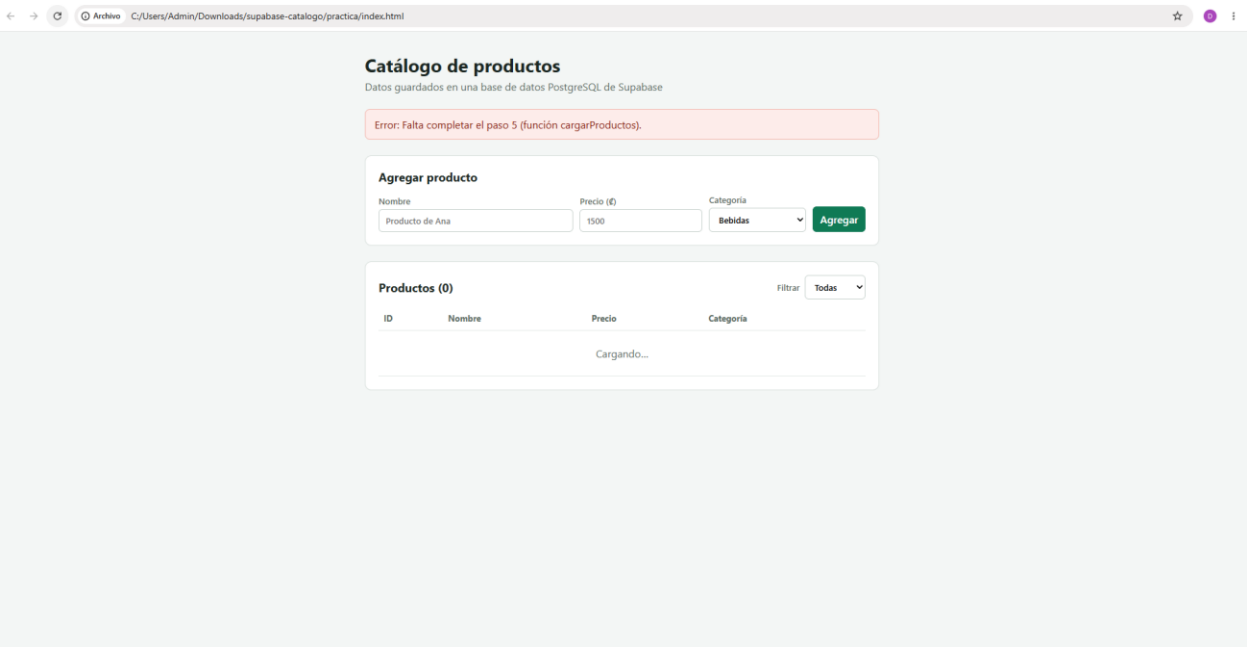
```
<script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2"></script>
<script>
  const SUPABASE_URL = 'https://xxxxxxxx.supabase.co';      // pegar la Project URL
  const SUPABASE_KEY = 'pegar-aquí-la-publishable-key';     // pegar la publishable key

  // No se llama "supabase" porque ese nombre ya lo usa la librería
  const cliente = window.supabase.createClient(SUPABASE_URL, SUPABASE_KEY);
```

Qué hace: la primera etiqueta <script> descarga la librería desde el CDN, tal como lo indica la documentación (Supabase, s. f.-d), y deja disponible el objeto global supabase. createClient crea el cliente que se usará en todas las consultas, con la dirección y la llave del proyecto.

Resultado esperado: guardar el archivo (Ctrl+S) y abrir practica/index.html con doble clic. La página carga y muestra el aviso rojo *Falta completar el paso 5*. Ese aviso es correcto: significa que la conexión está configurada y que falta escribir la consulta.

Nunca debe pegarse la *secret key* en la página. Esa llave se salta todas las políticas de seguridad (Supabase, s. f.-a).



Paso 5. Consultar y mostrar los productos

Qué se va a hacer: pedir a Supabase todas las filas de la tabla y mostrarlas.

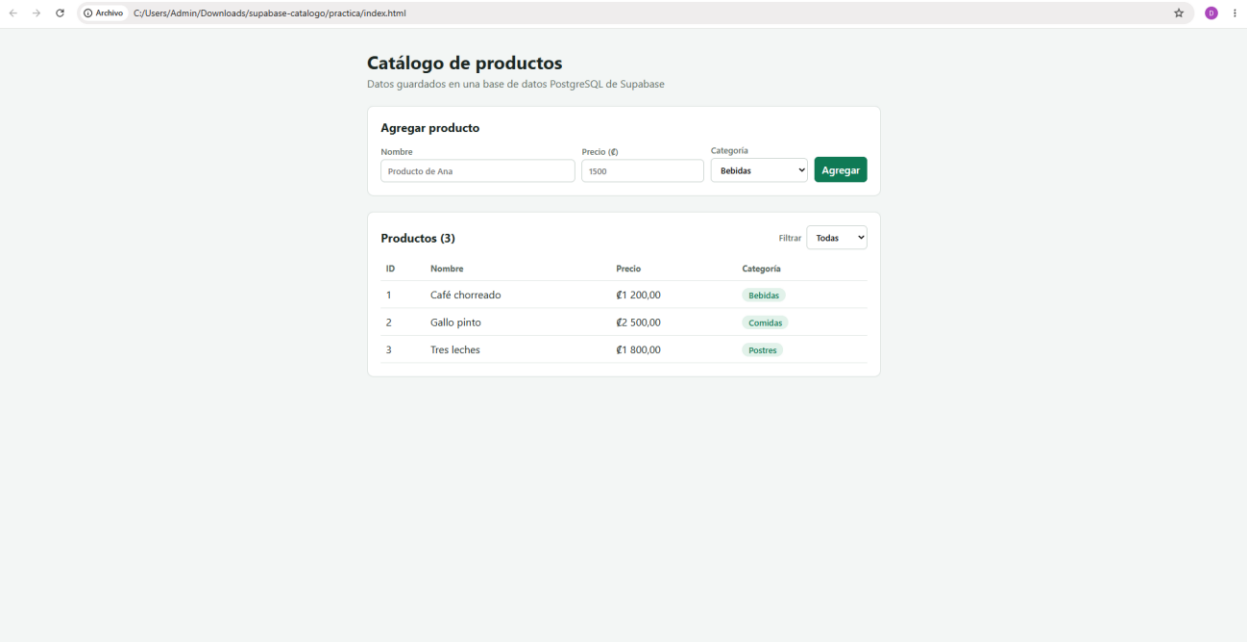
Archivo: practica/index.html, función cargarProductos. Borrar la línea mostrarError('Falta completar el paso 5...') y escribir en su lugar el contenido de esta función:

```
async function cargarProductos() {
  const { data, error } = await cliente
    .from('productos')
    .select('*')
    .order('id');

  if (error) { mostrarError(error); return; }
  pintarTabla(data);
}
```

Qué hace: from('productos') indica la tabla, select('*') pide todas las columnas y order('id') las ordena por id. Equivale a select * from productos order by id. Como la petición viaja por internet, se usa await para esperar la respuesta. La respuesta trae data, con las filas, o error si algo falló; en el primer caso pintarTabla las dibuja.

Resultado esperado: guardar y recargar la página (F5). La tabla muestra los tres productos y el título dice *Productos (3)*.



Paso 6. Insertar un producto desde el formulario

Qué se va a hacer: guardar en Supabase el producto que se escribe en el formulario.

Archivo: practica/index.html, función agregarProducto. Borrar la línea del aviso del paso 6 y completar la función así:

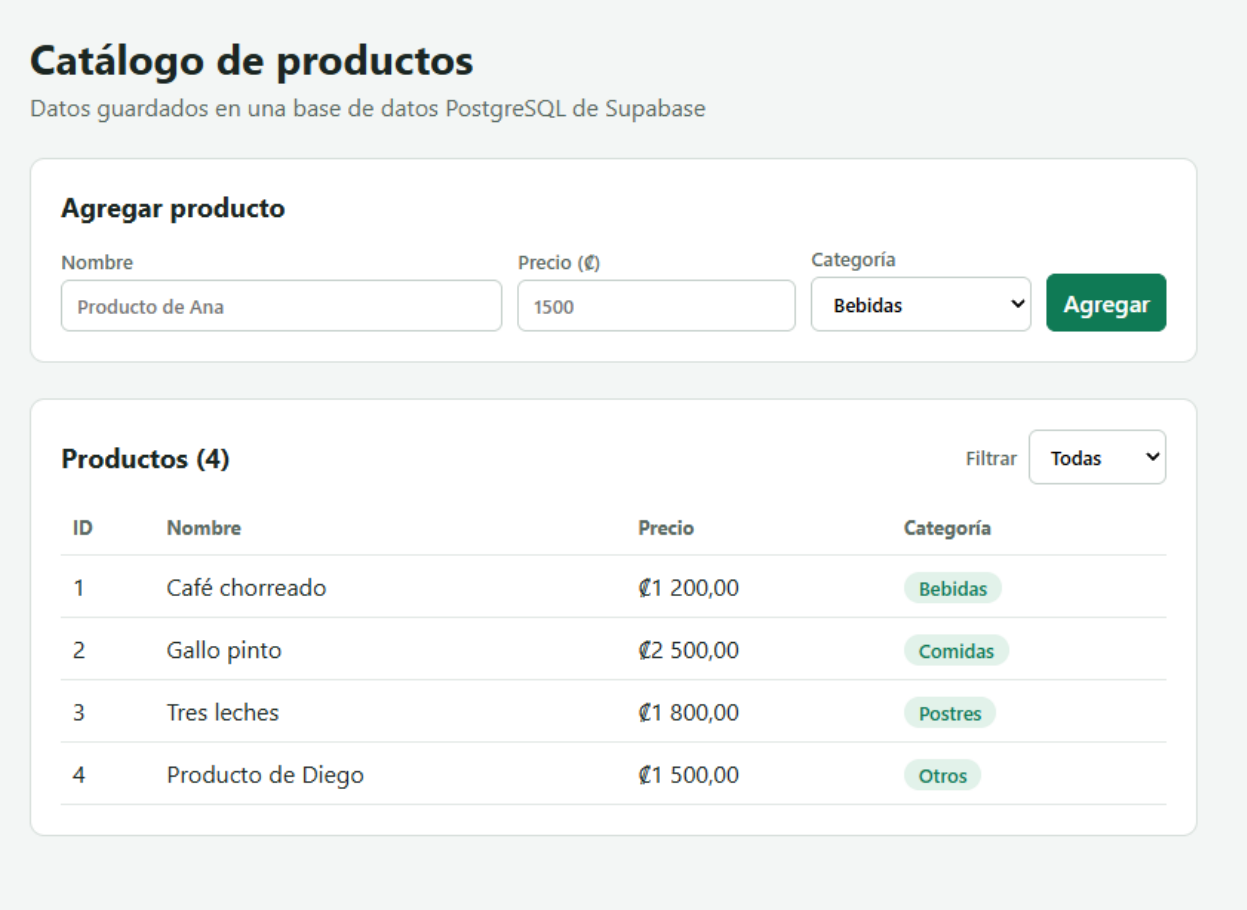
```
async function agregarProducto(evento) {
 evento.preventDefault();
 const nuevo = {
  nombre: formulario.nombre.value,
  precio: Number(formulario.precio.value),
  categoria: formulario.categoria.value
 };
```

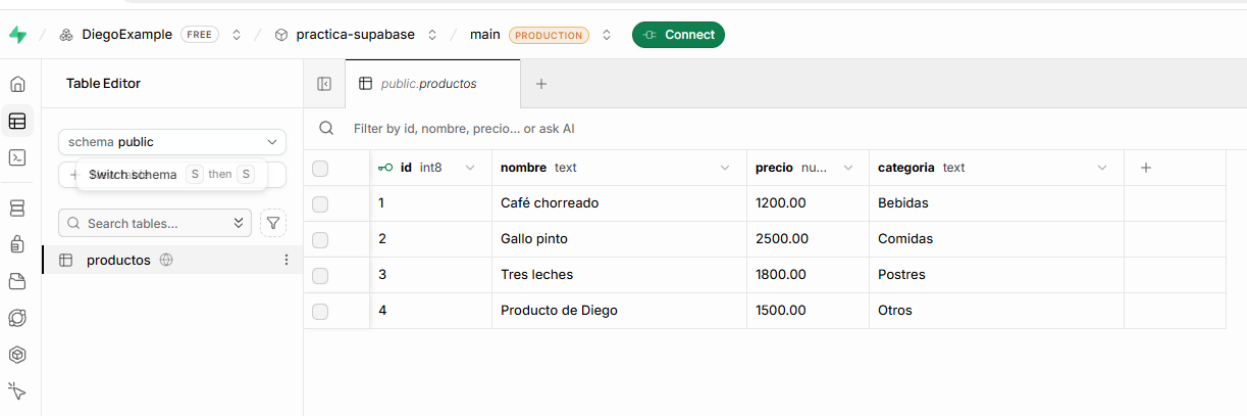
```
  const { error } = await cliente.from('productos').insert(nuevo);

  if (error) { mostrarError(error); return; }
  formulario.reset();
  cargarProductos();
}
```

Qué hace: preventDefault evita que el formulario recargue la página. El objeto nuevo reúne los tres datos del formulario; el id no se envía porque PostgreSQL lo genera. insert(nuevo) equivale a insert into productos (...) values (...). Si todo sale bien, se limpia el formulario y se vuelve a consultar la tabla para mostrar el producto nuevo.

Resultado esperado: escribir un producto, por ejemplo *Producto de Ana* con precio 1500 y categoría Otros, y hacer clic en *Agregar*. El producto aparece en la tabla con id 4, y también en el Table Editor de Supabase.





Paso 7. Filtrar por categoría (opcional)

Qué se va a hacer: consultar solo los productos de una categoría, usando el selector *Filtrar* que ya trae la página.

Archivo: practica/index.html. Reemplazar toda la función cargarProductos por esta versión:

```
async function cargarProductos() {
```

```
  let consulta = cliente
    .from('productos')
    .select('*')
    .order('id');

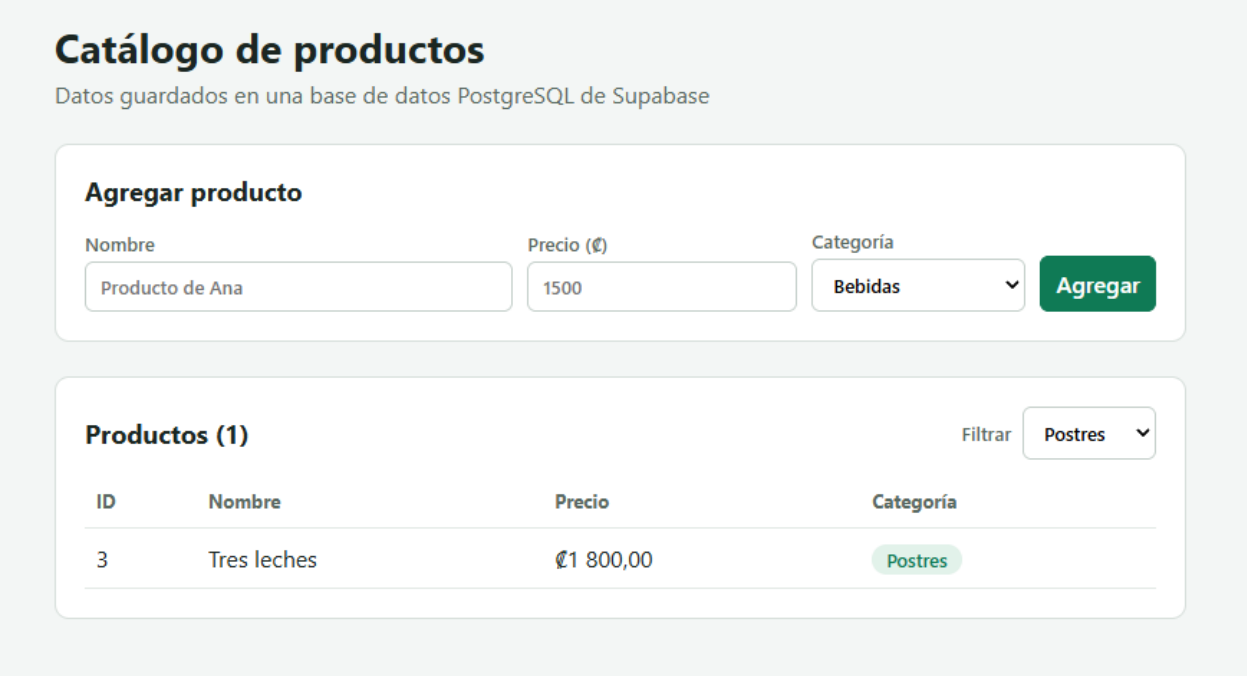
  if (selectCategoria.value !== 'Todas') {
    consulta = consulta.eq('categoria', selectCategoria.value);
  }

  const { data, error } = await consulta;

  if (error) { mostrarError(error); return; }
  pintarTabla(data);
}
```

Qué hace: si el selector tiene una categoría distinta de *Todas*, se agrega .eq('categoria', ...) a la consulta, que equivale a where categoria = 'Postres'. El cambio del selector ya está conectado a cargarProductos, así que no hace falta otro botón.

Resultado esperado: al elegir *Postres* solo quedan Tres leches y los postres que se hayan agregado.

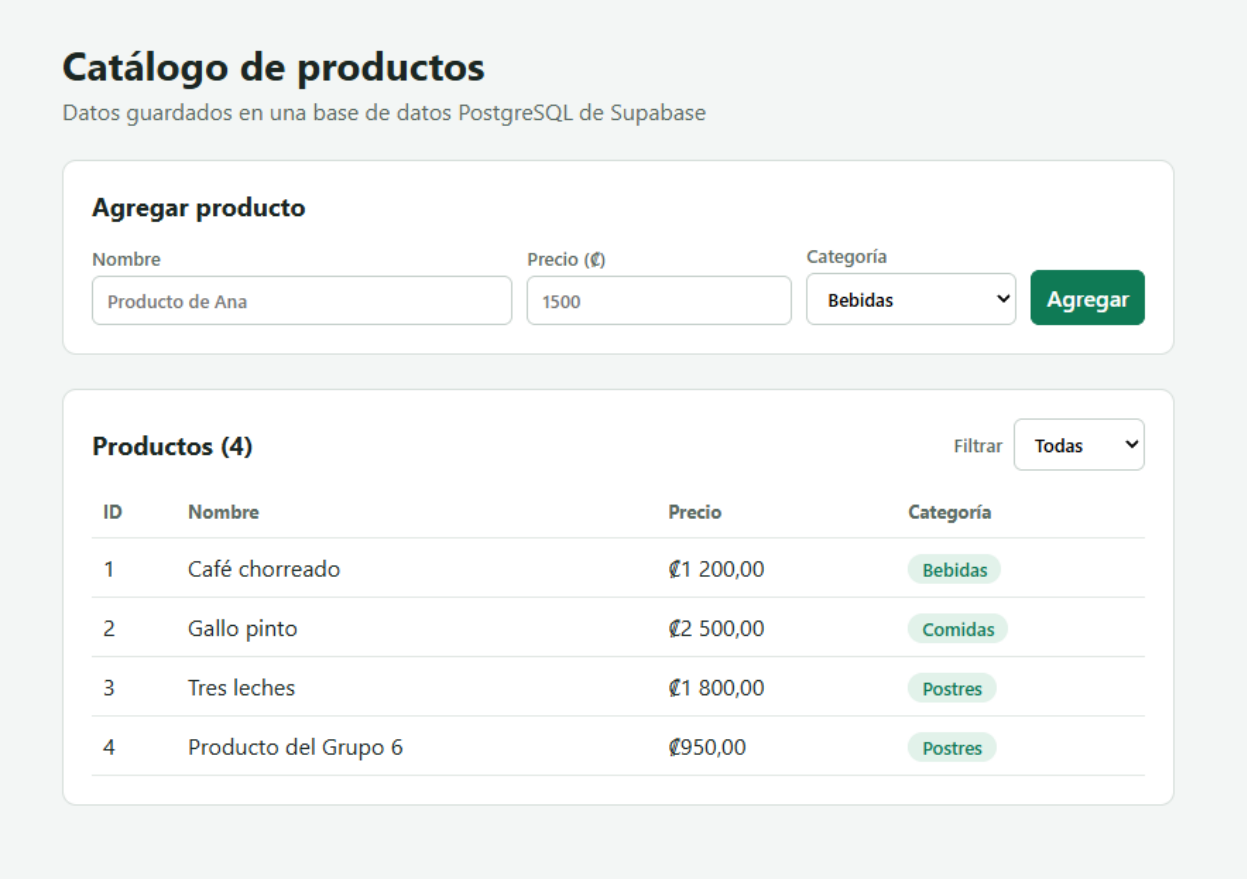

Catálogo de productos
Datos guardados en una base de datos PostgreSQL de Supabase
Agregar producto
Nombre
Producto de Ana
Precio (₡)
1500
Categoría
Bebidas
Agregar
Productos (1)
Filtrar
Postres
ID
Nombre
Precio
Categoría
3
Tres leches
₡1 800,00
Postres

6. Resultado final

El catálogo cumple lo que se definió al inicio. La página consulta la tabla productos en Supabase y la muestra; inserta productos nuevos que quedan guardados en PostgreSQL y se ven tanto en la página como en el panel; y filtra por categoría con una consulta a la API. Los precios se muestran en colones con el formato de Costa Rica.

Si algo falla, la página muestra el mensaje de error de Supabase junto con una pista: falta el grant, falta una política RLS, la llave está mal o la dirección no responde.

La copia de final/index.html que está en el repositorio ya viene conectada al proyecto del grupo en Supabase (catalogo-grupo6), así que funciona apenas se abre, sin configurar nada. Durante la práctica guiada sirve de respaldo para quien no logre crear su propio proyecto.



Para comprobarlo de punta a punta se puede agregar un producto, recargar la página y verificar que sigue ahí: los datos no están en el navegador sino en la base de datos.

7. Problemas encontrados

7.1 El nombre «supabase» ya estaba ocupado

Qué sucedió: al escribir la conexión, la forma más directa de nombrar el cliente era const supabase = supabase.createClient(...), con el mismo nombre de la librería.

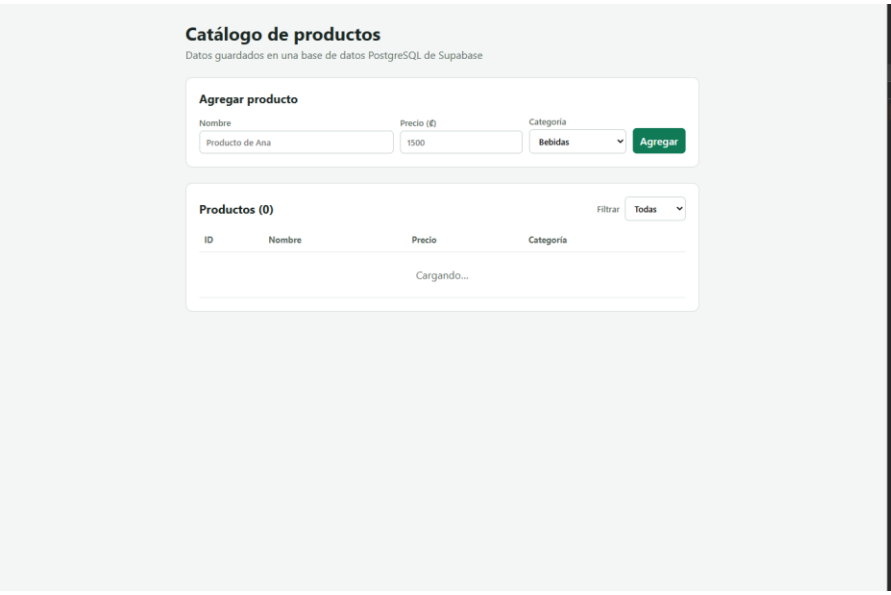
Cuál fue el error: el navegador no ejecuta el script y la consola muestra Uncaught SyntaxError: Identifier 'supabase' has already been declared.

En qué momento ocurrió: al configurar la conexión (paso 4).

Cómo investigamos la causa: descargamos el archivo que entrega el CDN (https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2, versión 2.118.0) y vimos que empieza con var supabase=. Esa línea crea una variable global llamada supabase. En JavaScript no se puede declarar con const un nombre que otro script ya declaró con var en el ámbito global.

Solución: llamar cliente a la variable y crearla con window.supabase.createClient(...). Dejamos un comentario en el código para que nadie la renombre.

Cómo comprobamos que quedó resuelto: la página carga sin errores en la consola y muestra los productos.





7.2 La tabla no quedaba disponible para la API

Qué sucedió: la primera versión del script SQL creaba la tabla, activaba RLS y definía las políticas, pero no incluía ningún grant.

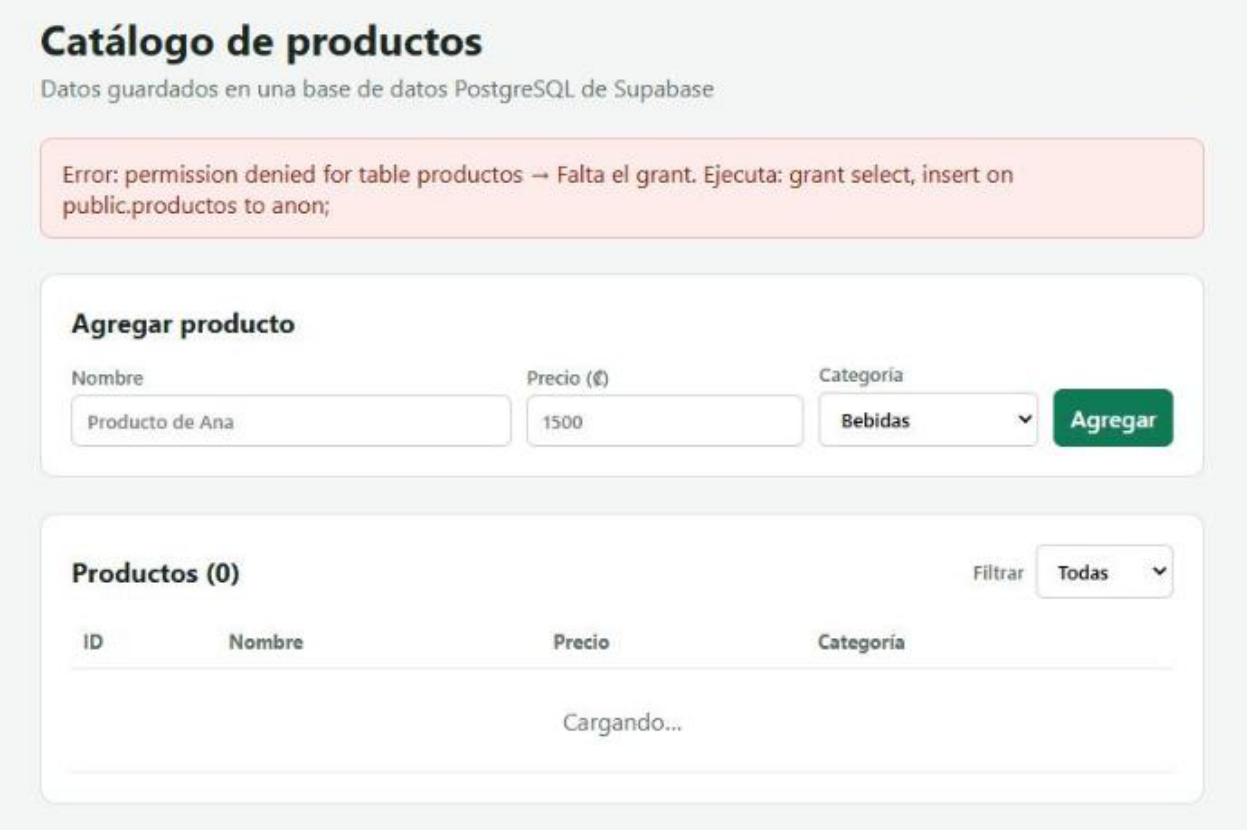
Cuál fue el error: con esa versión, la API responde permission denied for table productos con el código 42501, y la página no muestra nada (Supabase, 2026).

En qué momento ocurrió: lo detectamos al verificar las fuentes de esta investigación y lo reprodujimos el 3 de noviembre de 2026, al crear el proyecto del grupo con la opción *Automatically expose new tables* desmarcada.

Cómo investigamos la causa: el registro de cambios de Supabase del 28 de abril de 2026 anuncia que las tablas nuevas del esquema public dejan de exponerse automáticamente a la API, y que el cambio se aplica a todos los proyectos desde el 30 de octubre de 2026 (Supabase, 2026). La documentación de seguridad confirma que una tabla no es alcanzable por la API si no se le dio privilegios a un rol (Supabase, s. f.-f).

Solución: agregar al script grant select, insert on public.productos to anon;, que da al rol de la página solo los dos permisos que usa.

Cómo comprobamos que quedó resuelto: ejecutamos el script sin el grant y la página mostró el error de la figura siguiente, con cero productos. Después ejecutamos solo la línea del grant, recargamos y la página mostró los tres productos y permitió agregar uno nuevo (figura de la sección 6). La inserción funcionó sin dar permisos adicionales sobre la secuencia que genera el id.



8. Conclusiones

Lo que más nos sorprendió fue cuánto trabajo se ahorra en el backend. El catálogo guarda y consulta datos reales sin que hayamos escrito un servidor, y todo el código de Supabase en la página son dos funciones cortas. Lo que sí hubo que aprender bien fue la seguridad: entender que la llave pública, los permisos grant y las políticas RLS son tres cosas distintas, y que si falla cualquiera de las tres la página simplemente no muestra nada.

La dificultad principal no fue el código sino la documentación desactualizada. El cambio de permisos del 30 de octubre de 2026 deja sin funcionar los tutoriales escritos antes de esa fecha. La lección para nosotros es leer el registro de cambios oficial antes de confiar en una guía de terceros.

Como ventaja nos quedamos con que es PostgreSQL: lo aprendido sobre SQL y bases de datos relacionales se aplica en cualquier otro proyecto. La limitación que más pesa para un estudiante es el plan gratuito, que pausa los proyectos tras una semana sin uso y no permite más de dos activos.

La usaríamos en un proyecto pequeño o mediano que necesite usuarios y datos sin un equipo dedicado al backend, por ejemplo un sistema de inventario o una aplicación móvil del curso. A otro estudiante le recomendaríamos empezar por el SQL Editor y el Table Editor antes de tocar código, y tener abierta la consola del navegador desde el primer minuto, porque ahí aparecen los mensajes de error de Supabase.

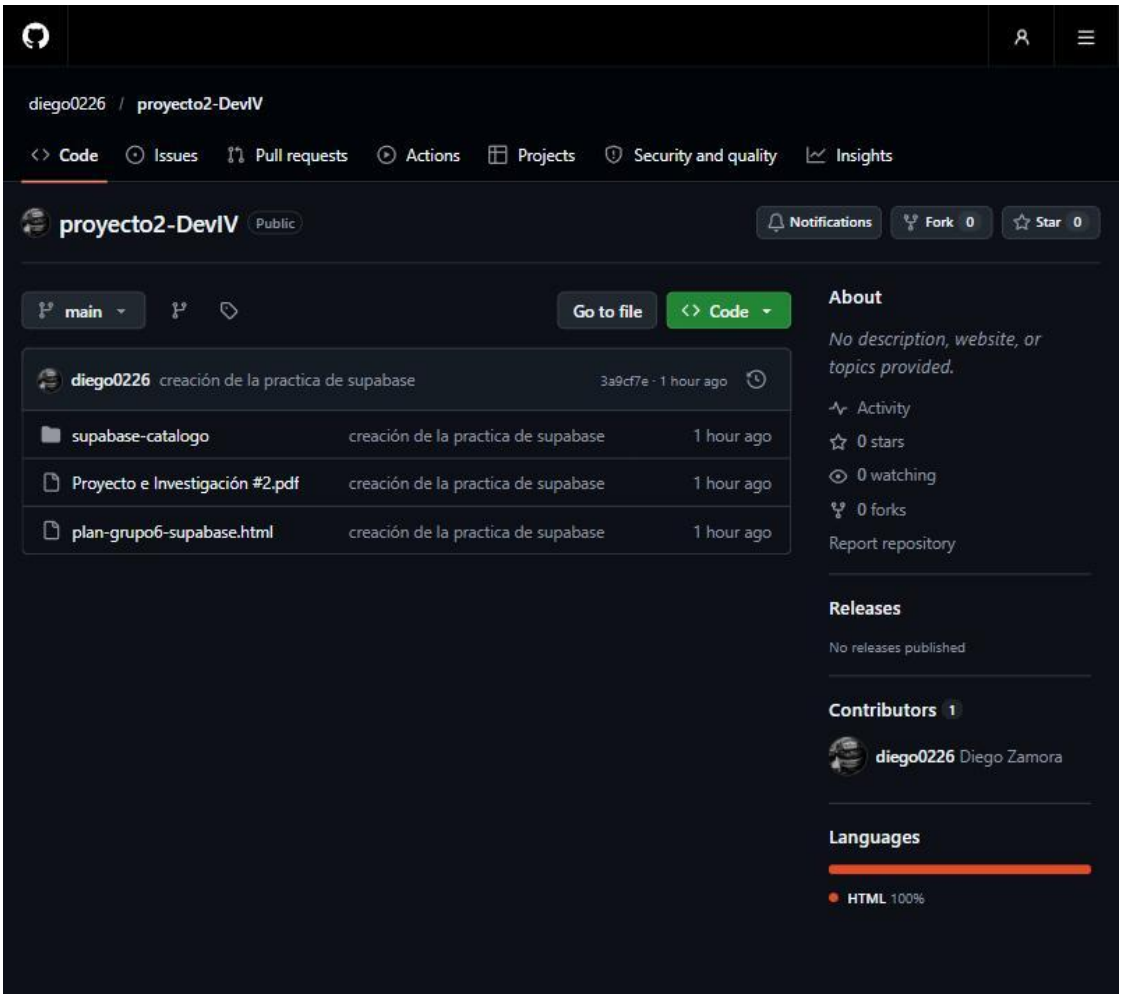
9. Repositorio Git

El proyecto está publicado en https://github.com/diego0226/proyecto2-DevIV, dentro de la carpeta supabase-catalogo. El archivo README.md incluye los requisitos, las instrucciones para ejecutarlo, los pasos de la práctica y una tabla de solución de problemas.

```
supabase-catalogo/
├── README.md              requisitos, ejecución, práctica y solución de problemas
├── sql/
│   └── 01_crear_tabla.sql  tabla, permisos, políticas RLS y datos de ejemplo
├── practica/
│   └── index.html         plantilla para la práctica guiada (con huecos TODO)
└── final/
    └── index.html         versión terminada
```

Para ejecutarlo no hace falta configurar nada: final/index.html ya está conectado al proyecto del grupo en Supabase y basta con abrirlo en el navegador. Para conectarlo a un proyecto propio se siguen los pasos 1, 2 y 4 de esta guía.

El código completo de la página terminada está en supabase-catalogo/final/index.html del repositorio.


diego0226 / proyecto2-DevIV
Code
Issues
Pull requests
Actions
Projects
Security and quality
Insights
proyecto2-DevIV
Public
Notifications
Fork 0
Star 0
main
Go to file
Code
diego0226 creación de la practica de supabase
3a9cf7e · 1 hour ago
supabase-catalogo
creación de la practica de supabase
1 hour ago
Proyecto e Investigación #2.pdf
creación de la practica de supabase
1 hour ago
plan-grupo6-supabase.html
creación de la practica de supabase
1 hour ago
About
No description, website, or topics provided.
Activity
0 stars
0 watching
0 forks
Report repository
Releases
No releases published
Contributors 1
diego0226 Diego Zamora
Languages
HTML 100%

10. Referencias

Bort, J. (2025, 3 de octubre). Supabase nabs $5B valuation, four months after hitting $2B. *TechCrunch*. https://techcrunch.com/2025/10/03/supabase-nabs-5b-valuation-four-months-after-hitting-2b/

Kakashvili, G. (2023). Implementation of backend services in mobile applications. *Intellect, 26*(2). https://inteleqti.openjournals.ge/index.php/intelecti/article/view/7968

Supabase. (s. f.-a). *API keys*. Supabase Docs. Recuperado el 3 de noviembre de 2026 de https://supabase.com/docs/guides/api/api-keys

Supabase. (s. f.-b). *Architecture*. Supabase Docs. Recuperado el 3 de noviembre de 2026 de https://supabase.com/docs/guides/getting-started/architecture

Supabase. (s. f.-c). *Data REST API*. Supabase Docs. Recuperado el 3 de noviembre de 2026 de https://supabase.com/docs/guides/api

Supabase. (s. f.-d). *JavaScript: Installing*. Supabase Docs. Recuperado el 3 de noviembre de 2026 de https://supabase.com/docs/reference/javascript/installing

Supabase. (s. f.-e). *Row Level Security*. Supabase Docs. Recuperado el 3 de noviembre de 2026 de https://supabase.com/docs/guides/database/postgres/row-level-security

Supabase. (s. f.-f). *Securing your API*. Supabase Docs. Recuperado el 3 de noviembre de 2026 de https://supabase.com/docs/guides/api/securing-your-api

Supabase. (s. f.-g). *supabase/supabase* [Repositorio de código]. GitHub. Recuperado el 3 de noviembre de 2026 de https://github.com/supabase/supabase

Supabase. (s. f.-h). *Supabase features*. Supabase Docs. Recuperado el 3 de noviembre de 2026 de https://supabase.com/docs/guides/getting-started/features

Supabase. (s. f.-i). *Supabase pricing*. Recuperado el 3 de noviembre de 2026 de https://supabase.com/pricing

Supabase. (2024, 15 de abril). *We are moving to General Availability*. https://supabase.com/ga

Supabase. (2026, 28 de abril). *Breaking change: Tables not exposed to Data and GraphQL API automatically*. Supabase Changelog. https://supabase.com/changelog/45329-breaking-change-tables-not-exposed-to-data-and-graphql-api-automatically

The PostgreSQL Global Development Group. (s. f.-a). *About*. Recuperado el 3 de noviembre de 2026 de https://www.postgresql.org/about/

The PostgreSQL Global Development Group. (s. f.-b). *5.9. Row security policies*. En PostgreSQL 18 documentation. Recuperado el 3 de noviembre de 2026 de https://www.postgresql.org/docs/current/ddl-rowsecurity.html